

TECHNICAL DOCUMENTATION

EventHub Selenium BDD Automation Framework

Code Structure & Architecture Guide — for client walkthrough

Project: selenium-bdd-framework

Stack: Java 17 · Selenium WebDriver · Cucumber BDD · JUnit 5 · Maven

Pattern: Page Object Model (POM)

Status: Verified — executed live against the target application

Table of Contents

1. Executive Summary
2. How a Test Runs — End-to-End Flow
3. Top-Level Folder Structure
4. `src/main/java` — Reusable Framework Code
5. `src/main/resources` — Configuration & Test Data
6. `src/test/java` — Test Logic (Cucumber Glue)
7. `src/test/resources` — Feature Files & Test Config
8. Build & Reporting Outputs
9. How to Run the Tests
10. Design Principles Applied

1. Executive Summary

This framework automates end-to-end testing of the **EventHub** web application (<https://eventhub.rahulshettyacademy.com>) using **Selenium WebDriver** for browser control and **Cucumber** for writing test scenarios in plain English (Gherkin syntax), so that both technical and non-technical stakeholders can read and understand what is being tested.

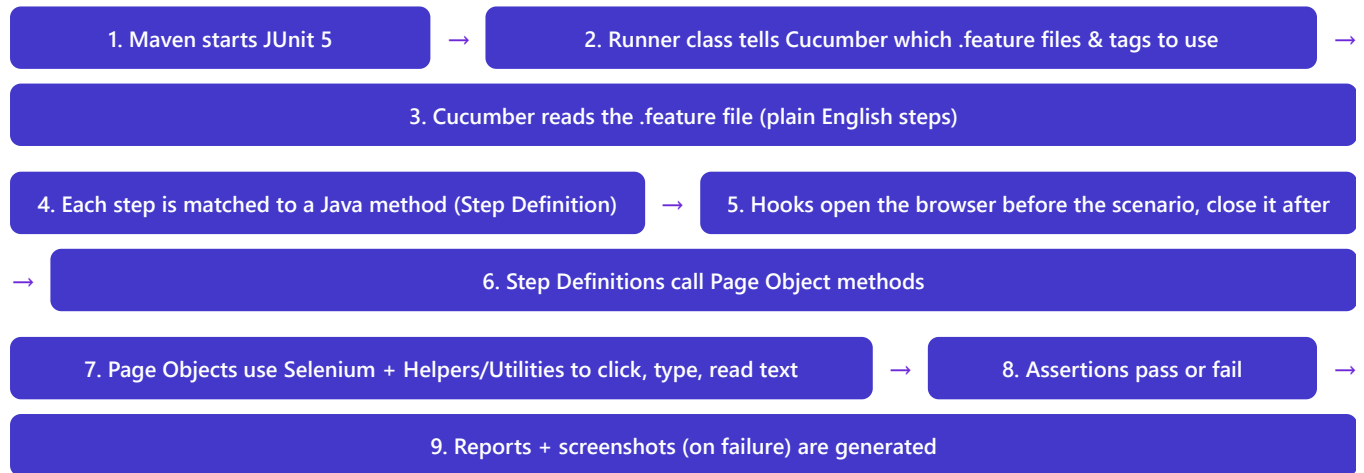
The codebase follows the **Page Object Model (POM)**: every screen of the application (Login, Events listing, Booking form, My Bookings, Booking Detail, Admin) has a corresponding Java class that knows how to interact with that screen. Test scenarios never talk to the browser directly — they call methods on these page objects, which keeps the tests short, readable, and easy to maintain when the UI changes.

Layer	Technology	Purpose
Language	Java 17	Core programming language
Browser automation	Selenium WebDriver 4	Drives the real browser (Chrome / Firefox / Edge)
Test scenarios	Cucumber (Gherkin)	Plain-English test scripts (<code>.feature</code> files)
Test runner	JUnit 5	Executes the Cucumber scenarios and reports results
Build tool	Maven	Compiles code, manages dependencies, runs suites
Driver management	WebDriverManager	Automatically downloads the correct browser driver
Logging	Log4j2 / SLF4J	Execution logs for debugging failures
Reporting	Allure, Extent, Cucumber HTML	Visual, shareable test result reports

Verified: This framework was executed live against the real application in a real browser. The core flow — create an event as an admin, book it as a user, and confirm the seat count drops by exactly one — passed end-to-end, along with the refund-eligibility checks.

2. How a Test Runs — End-to-End Flow

When someone runs a command like `mvn test -Dtest=SmokeTestRunner`, this is what happens, in order:



This layered flow is the reason the framework is maintainable: a change to the application's UI usually only requires updating one Page Object class, not every test scenario that uses that screen.

3. Top-Level Folder Structure

selenium-bdd-framework/	
— src/main/java/	→ Reusable framework code (pages, helpers, utilities, config)
— src/main/resources/	→ Configuration files and test data (non-code)
— src/test/java/	→ Test-specific code (Cucumber glue: steps, hooks, runners)
— src/test/resources/	→ Feature files (the actual test scenarios) and test config
— reports/	→ Generated after each run: Cucumber HTML/JSON report
— screenshots/	→ Auto-captured screenshots of any FAILED scenario
— logs/	→ Execution log files (Log4j2)
— allure-results/	→ Raw data consumed by the Allure reporting tool
— allure-report/	→ Generated Allure HTML report (via `mvn allure:report`)
— pom.xml	→ Maven project file: dependencies, plugins, build profiles
— README.md	→ Setup and execution instructions
— .gitignore	→ Files/folders excluded from version control

Folder	Contains code?	Purpose
src/main/java	Yes	The reusable "engine" of the framework — would exist even without any tests written yet
src/main/resources	No (config/data)	Settings and test data files read by the code above
src/test/java	Yes	Glue code that connects plain-English scenarios to the engine above
src/test/resources	No (Gherkin/config)	The actual test scenarios, written in plain English, plus test-only settings
reports / screenshots / logs / allure-*	No (generated output)	Created automatically every time tests run; safe to delete and regenerate

4. src/main/java — Reusable Framework Code

Everything in this folder is **infrastructure** — code that supports testing but contains no test logic itself. It is organized into 6 packages (sub-folders), each with one job:

4.1 base/ — the foundation every screen builds on

`com/eventhub/framework/base/BasePage.java`

The parent class for every Page Object. Holds the shared browser connection and generic actions (navigate to a page, read the current URL/title). Every screen-specific class (LoginPage, EventsPage, etc.) extends this one, so common behavior is written once.

4.2 config/ — application settings

`com/eventhub/framework/config/ConfigManager.java`

Reads `config.properties` and the active environment file (qa/dev/staging), and makes settings like the website URL, browser choice, and wait times available to the rest of the code. Also allows any setting to be overridden from the command line (e.g. `-Dbrowser=firefox`) without touching any file.

4.3 constants/ — fixed values used across the project

File	What it holds
<code>AppConstants.java</code>	Page routes (/login, /events, /bookings), default timeout values, folder paths
<code>TestDataConstants.java</code>	Names of the test-data JSON files and the "profile" keys inside them

4.4 factory/ — starting and stopping the browser

File	Responsibility
<code>BrowserFactory.java</code>	Creates a Chrome, Firefox, or Edge browser instance with the right settings (window size, headless mode, etc.). Includes a fallback to use a manually-provided driver file if automatic download is blocked by a network/firewall.
<code>DriverManager.java</code>	Keeps track of the browser instance for each test <i>thread</i> , so that multiple tests can run in parallel without interfering with each other's browser windows.

4.5 helpers/ — reusable actions that aren't tied to one screen

File	Responsibility
<code>UIActionsHelper.java</code>	The most-used file in the project. Provides safe click/type/read-text/wait-until-visible actions used by every Page Object, including an automatic retry if a normal click is blocked by an overlay.
<code>NavigationHelper.java</code>	Builds page URLs from the configured base address and handles browser back/forward/refresh.

AuthenticationHelper.java	The reusable "log in" routine — enters the email/password and confirms login succeeded. Called at the start of almost every scenario.
AlertHelper.java	Handles browser pop-up alerts/confirmations (accept, dismiss, read text).
DropdownHelper.java	Selects options from standard drop-down menus.
FrameHelper.java	Switches in and out of embedded page frames (iframes), if the application uses them.
WindowHelper.java	Handles pop-up browser tabs/windows (e.g. switching to a new tab).
ApiHelper.java	Optional helper for calling backend APIs directly, for setup/verification without the UI.

4.6 pages/ — one class per application screen

This is the heart of the Page Object Model. Each class below models exactly one screen of the application and exposes plain-English methods for interacting with it.

File	Screen it represents	Example methods
LoginPage.java	/login	enterEmail, enterPassword, clickLogin, isBrowseEventsLinkVisible
AdminEventPage.java	/admin/events (create event form)	createEvent, isEventCreatedToastVisible
EventsPage.java	/events (events listing)	findCardByTitle, getSeatsRemaining, clickBookNowOnCard
BookingPage.java	Book-tickets form	fillBookingForm, setTicketCount, clickConfirmBooking, getBookingReference
MyBookingsPage.java	/bookings (booking history)	findCardByBookingRef, clickFirstViewDetails
BookingDetailPage.java	/bookings/{id} (single booking + refund check)	clickCheckRefundEligibility, getRefundResultText

Why this matters to the client: if EventHub changes the design of, say, the booking form, a developer only needs to update `BookingPage.java`. Every test scenario that books a ticket keeps working without being rewritten.

4.7 utilities/ — small, generic helper functions

File	Responsibility
WaitUtils.java	Explicit waits ("wait until visible/clickable") — avoids unreliable fixed pauses (Thread.sleep)
JSUtils.java	Runs JavaScript in the browser for the rare cases native clicks/typing aren't reliable enough
ScreenshotUtils.java	Captures and saves a screenshot, used automatically when a scenario fails
LogUtils.java	Standard logger used by every class for consistent log output
DateTimeUtils.java	Generates dates/timestamps, including a "future date" generator for event scheduling
FileUtils.java	Generic file read/write helpers
ExcelUtils.java	Reads test data from Excel spreadsheets, if needed for data-driven testing
JsonUtils.java	Reads test data from JSON files (used for event/booking test data)

PropertyReaderUtils.java	Low-level reader for .properties configuration files
EnvironmentUtils.java	Determines which environment (qa/dev/staging) is active for the current run

5. src/main/resources — Configuration & Test Data

`config.properties`

Default settings for every run: which browser to use, timeout durations, whether to run headless (no visible window), and paths for logs/screenshots/reports.

`environments/qa.properties` · `dev.properties` · `staging.properties`

Environment-specific values — the website address and the login credentials to use for each environment. Switching environments is a single command-line flag (`-Denv=staging`), no code changes required.

`log4j2.xml`

Configures how logs are formatted and where they're written (console + a rolling daily log file under `logs/`).

`testdata/eventTestData.json` & `bookingTestData.json`

Test input values kept separate from the test logic — the event description/city/venue/price used when creating a test event, and the customer name/email/phone used when booking, plus the expected refund-eligibility messages. A client or QA lead can update these values without touching any Java code.

6. src/test/java — Test Logic (Cucumber Glue)

Everything here is specific to *running the tests* — as opposed to `src/main/java`, which would exist even in a library with no tests. This is organized into 4 packages:

6.1 context/ — sharing data between steps

`com/eventhub/framework/context/ScenarioContext.java`

A small in-memory store that carries information from one test step to the next within the same scenario — for example, remembering the "seats before booking" count so it can be compared against "seats after booking" later in the same test.

6.2 hooks/ — setup and teardown around every scenario

`com/eventhub/framework/hooks/Hooks.java`

Runs automatically **before** and **after** every scenario:

- **Before:** opens a fresh browser window
- **After:** if the scenario failed, takes a screenshot and captures the browser's console log, attaches both to the reports, then always closes the browser cleanly

6.3 stepdefinitions/ — translating English into actions

Each line of a `.feature` file (e.g. "When I click Book Now on the matched event card") is matched to exactly one Java method below, which then calls the appropriate Page Object.

File	Covers
<code>CommonSteps.java</code>	Login, page navigation, generic visibility/URL checks reused across every feature
<code>AdminEventSteps.java</code>	Creating a new event from the admin panel
<code>EventBookingSteps.java</code>	Finding event cards, reading seat counts, filling and confirming the booking form
<code>RefundEligibilitySteps.java</code>	Viewing booking details and checking refund eligibility (spinner + result)

6.4 runners/ — which scenarios to run, and how

File	Runs scenarios tagged	Typical use
<code>SmokeTestRunner.java</code>	<code>@Smoke</code>	Fast check of the most critical business flow before every deploy
<code>RegressionTestRunner.java</code>	<code>@Regression</code>	Full coverage — run before a release
<code>SanityTestRunner.java</code>	<code>@Sanity</code>	Very basic "is the app up" checks

7. src/test/resources — Feature Files & Test Config

This folder contains the parts a **non-developer can read and review** — the actual test scripts, written in plain English (Gherkin), organized by suite:

File	Suite	What it tests
features/smoke/event_booking.feature	Smoke	Create a new event as admin → book it as a user → confirm the seat count drops by exactly 1
features/regression/refund_eligibility.feature	Regression	Book with 1 ticket → "Eligible for refund"; book with 3 tickets → "Not eligible for refund"
features/sanity/login_sanity.feature	Sanity	Basic login and events-page rendering checks

junit-platform.properties

Wires up Cucumber's reporting plugins (Cucumber HTML/JSON, Allure, Extent) and enables running scenarios in parallel to save time.

allure.properties · extent.properties · extent-config.xml

Configuration for the Allure and Extent reporting tools (output folder locations, report title/theme).

8. Build & Reporting Outputs

These folders are **generated automatically** every time the tests run — nothing here needs to be written or maintained by hand, and they are safe to delete between runs.

Folder	Content
<code>reports/</code>	Cucumber HTML report + JSON report + Extent Spark report
<code>screenshots/</code>	One PNG per failed scenario, named after the scenario and timestamped
<code>logs/</code>	Full execution log (<code>automation.log</code>), rotated daily
<code>allure-results/</code>	Raw result files consumed by the Allure report generator
<code>allure-report/</code>	The final Allure HTML report, generated via <code>mvn allure:report</code>

9. How to Run the Tests

```
# Run everything
mvn test

# Run one suite specifically
mvn test -Dtest=SmokeTestRunner
mvn test -Dtest=RegressionTestRunner
mvn test -Dtest=SanityTestRunner

# Choose a browser / environment
mvn test -Dtest=SmokeTestRunner -Dbrowser=firefox
mvn test -Dtest=RegressionTestRunner -Denv=staging -Dheadless=true

# Generate and open the Allure report after a run
mvn allure:serve
```

Full setup instructions (prerequisites, configuration, extending the framework) are documented in [README.md](#) at the project root.

10. Design Principles Applied

- **Page Object Model:** UI locators and page actions live only in `pages/` ; test logic never touches Selenium directly.
- **No hard-coded waits:** every wait is an explicit, condition-based wait (`WaitUtils`) instead of an unreliable fixed pause.
- **Externalized configuration:** URLs, credentials, and timeouts live in properties files, not in code — changing environments needs no code changes.
- **Externalized test data:** input values live in JSON files, separate from test logic, so QA/business users can adjust scenarios' data without touching Java.
- **Thread-safe parallel execution:** each test thread gets its own isolated browser session via `DriverManager` , so suites can run scenarios concurrently without cross-talk.
- **Automatic diagnostics on failure:** every failed scenario automatically captures a screenshot and browser console log, attached directly to the reports — no manual reproduction needed to start investigating.
- **CI/CD ready:** the same Maven commands used locally work unchanged in Jenkins, GitHub Actions, or Azure DevOps pipelines.

EventHub Selenium BDD Automation Framework — Code Structure & Architecture Guide. Generated to support client walkthroughs of the automation codebase.